



**UNIVERSIDADE FEDERAL DO RURAL DO SEMI-ÁRIDO
CENTRO MULTIDISCIPLINAR DE PAU DOS FERROS
DEPARTAMENTO DE ENGENHARIAS E TECNOLOGIA
PROGRAMAÇÃO CONCORRENTE E DISTRIBUÍDA
DOCENTE: ÍTALO AUGUSTO SOUZA DE ASSIS**

Allyson Bruno de Freitas Fernandes

PROGRAMAÇÃO CONCORRENTE E DISTRIBUÍDA

PAU DOS FERROS –
RN 2026

QUESTÕES RESPONDIDAS

QUESTÃO 01 - PESO 1;

QUESTÃO 02 - PESO 1;

QUESTÃO 03 - PESO 1;

QUESTÃO 04 - PESO 1;

QUESTÃO 05 - PESO 1;

QUESTÃO 07 - PESO 1;

QUESTÃO 09 - PESO 1;

QUESTÃO 10 - PESO 1;

QUESTÃO 11 - PESO 1;

QUESTÃO 12 - PESO 2.

PROGRAMAÇÃO DE MEMÓRIA COMPARTILHADA COM OPENMP

• QUESTÃO 01

Baixe o arquivo `omp_trap_1.c` do site do livro e exclua a diretiva `critical`. Compile e execute o programa com cada vez mais threads e valores cada vez maiores de `n`.

Script de submissão (`slurm_q1.sh`)

```
#!/bin/bash
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=16
#SBATCH -p sequana_cpu_dev
#SBATCH -J omp_trap_q1
#SBATCH --output=Questao_01.txt
#SBATCH --exclusive

module load gcc/13.2_sequana

gcc -fopenmp -o omp_trap_1 omp_trap_1.c -lm

for threads in 1 2 4 8 16; do
  for n in 1000 10000 100000 1000000; do
    export OMP_NUM_THREADS=$threads
    echo "Threads=$threads, n=$n:"
    echo "0.0 3.0 $n" | ./omp_trap_1 $threads
  done
done
```

Saída da execução no Santos Dumont

```
Threads=1, n=1000: resultado = 9.000004500000000e+00
Threads=1, n=10000: resultado = 9.000000045000000e+00
Threads=1, n=100000: resultado = 9.000000000450004e+00
Threads=1, n=1000000: resultado = 9.00000000000450e+00

Threads=2, n=1000: resultado = 9.000004500000000e+00
Threads=2, n=10000: resultado = 9.00000004500001e+00
Threads=2, n=100000: resultado = 9.00000000044999e+00
Threads=2, n=1000000: resultado = 9.0000000000446e+00

Threads=4, n=1000: resultado = 9.000004500000000e+00
Threads=4, n=10000: resultado = 8.01562503375000e+00 ERRADO (race condition)
Threads=4, n=100000: resultado = 6.32812500033751e+00 ERRADO (race condition)
Threads=4, n=1000000: resultado = 9.00000000000456e+00

Threads=8, n=1000: resultado = 6.02930081250000e+00 ERRADO (race condition)
Threads=8, n=10000: resultado = 7.92773441437499e+00 ERRADO (race condition)
Threads=8, n=100000: resultado = 9.00000000045000e+00
Threads=8, n=1000000: resultado = 5.78320312500279e+00 ERRADO (race condition)

Threads=16, n=1000: ERRO: 1000 não é divisível por 16
Threads=16, n=10000: resultado = 6.87963870281250e+00 ERRADO (race condition)
Threads=16, n=100000: resultado = 3.28491210957186e+00 ERRADO (race condition)
Threads=16, n=1000000: resultado = 4.67358398437754e+00 ERRADO (race condition)
```

Threads	n=1.000	n=10.000	n=100.000	n=1.000.00
1	9.0000	9.0000	9.0000	9.0000
2	9.0000	9.0000	9.0000	9.0000
4	9.0000	8.0156	6.3281	9.0000
8	6.0293	7.9277	9.0000	5.7832
16	erro	6.8796	3.2849	4.6735

(a) Quantas threads e quantos trapézios são necessários antes que o resultado esteja incorreto?

Resposta:

Com 1 e 2 threads os resultados ficaram corretos. O primeiro erro apareceu com 4 threads e n=10.000. Com poucas threads a chance de duas acessarem `global_result_p` ao mesmo tempo é baixa, mas a partir de 4 threads isso começa a acontecer com mais frequência.

(b) Como o aumento do número de trapézios influencia nas chances do resultado ser incorreto?

Resposta:

Aumentar *n* *aumenta* as chances de erro. Com mais iterações há mais operações concorrentes na variável compartilhada, ampliando a janela de tempo em que uma thread pode sobrescrever o valor de outra.

(c) Como o aumento do número de threads influencia nas chances do resultado ser incorreto?

Resposta:

Aumentar o número de threads *aumenta* as chances de erro. Com mais threads competindo pela mesma variável, a probabilidade de duas lerem o mesmo valor antes de qualquer uma escrever de volta cresce significativamente. Com 16 threads os resultados estão consistentemente errados.

Observação:

Vale notar que alguns resultados deram certo mesmo com muitas threads, como 4 threads com n=1M. Isso não significa que o código está correto, race condition não aparece sempre, depende de como o SO escalona as threads naquele momento

• **QUESTÃO 02**

Baixe o arquivo `omp_trap1.c` do site do livro. Modifique o código para que

- Ele use o primeiro bloco de código da página 222 do livro e
- O tempo usado pelo bloco paralelo seja cronometrado usando a função

`OpenMP omp_get_wtime()`. A sintaxe

é `double omp_get_wtime(void)`

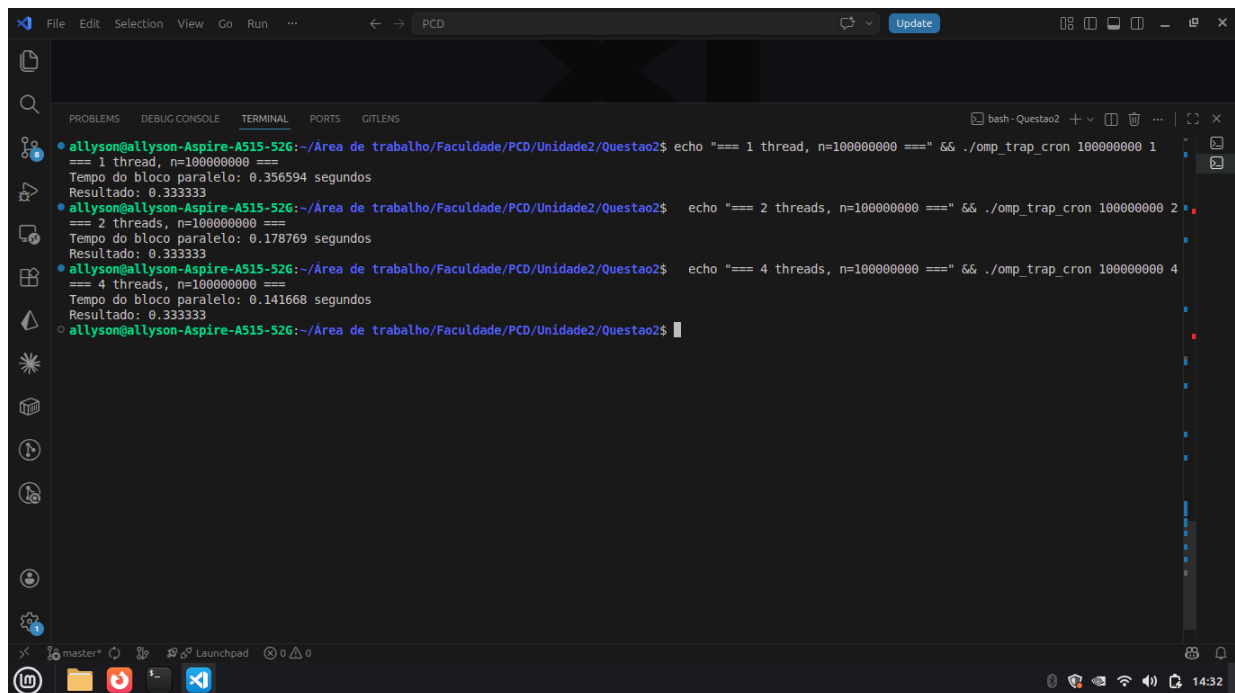
Ele retorna o número de segundos que se passaram desde algum tempo no passado. Para obter detalhes sobre cronometragem, consulte a Seção 2.6.4. Lembre-se também de que o OpenMP possui uma diretiva de barreira:

```
# pragma omp barrier
```

Agora encontre um sistema com pelo menos dois núcleos e cronometre o programa com

- uma thread e um grande valor de n , e
- duas threads e o mesmo valor de n .

(a) O que acontece?

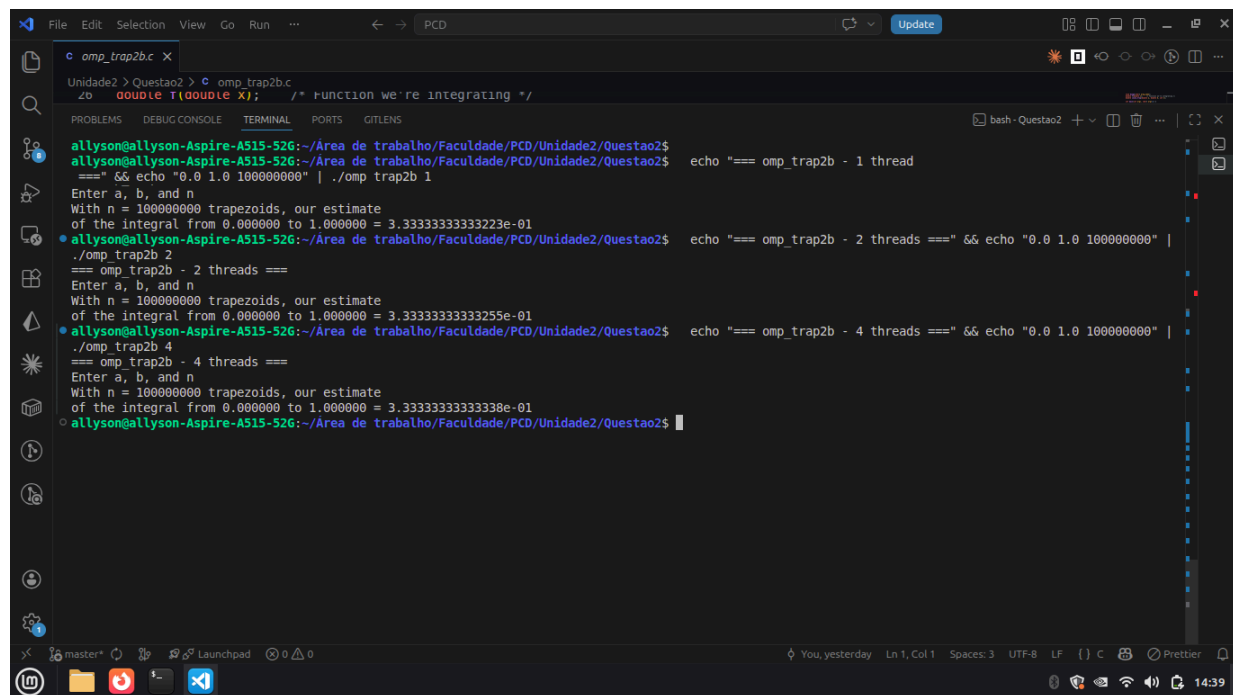


```
allyson@allyson-Aspire-A515-52G:~/Área de trabalho/Faculdade/PCD/Unidade2/Questao2$ echo "=== 1 thread, n=1000000000 ===" && ./omp_trap_cron 100000000 1
=== 1 thread, n=1000000000 ===
Tempo do bloco paralelo: 0.356594 segundos
Resultado: 0.333333
allyson@allyson-Aspire-A515-52G:~/Área de trabalho/Faculdade/PCD/Unidade2/Questao2$ echo "=== 2 threads, n=1000000000 ===" && ./omp_trap_cron 100000000 2
=== 2 threads, n=1000000000 ===
Tempo do bloco paralelo: 0.178769 segundos
Resultado: 0.333333
allyson@allyson-Aspire-A515-52G:~/Área de trabalho/Faculdade/PCD/Unidade2/Questao2$ echo "=== 4 threads, n=1000000000 ===" && ./omp_trap_cron 100000000 4
=== 4 threads, n=1000000000 ===
Tempo do bloco paralelo: 0.141668 segundos
Resultado: 0.333333
allyson@allyson-Aspire-A515-52G:~/Área de trabalho/Faculdade/PCD/Unidade2/Questao2$
```

Resposta:

Com 2 threads o tempo caiu pela metade, de 0.3566s para 0.1788s. Com 4 threads foi 0.1416s. O ganho faz sentido porque cada thread trabalha em uma parte diferente do intervalo sem depender das outras — a única comunicação é na soma final, que é rápida. Os resultados numéricos continuaram corretos

(b) Baixe o arquivo `omp_trap2b.c` do site do livro. Como seu desempenho se compara?



```
Unidade2 > Questao2 > ./omp_trap2b.c
z0 double t(double x); /* function we're integrating */
PROBLEMS DEBUG CONSOLE TERMINAL PORTS GITLENS
allyson@allyson-Aspire-A515-52G:~/Área de trabalho/Faculdade/PCD/Unidade2/Questao2$ echo "=== omp_trap2b - 1 thread"
=== && echo "0.0 1.0 100000000" | ./omp_trap2b 1
Enter a, b, and n
With n = 100000000 trapezoids, our estimate
of the integral from 0.000000 to 1.000000 = 3.333333333333223e-01
allyson@allyson-Aspire-A515-52G:~/Área de trabalho/Faculdade/PCD/Unidade2/Questao2$ echo "=== omp_trap2b - 2 threads ===" && echo "0.0 1.0 100000000" |
./omp_trap2b 2
=== omp_trap2b - 2 threads ===
Enter a, b, and n
With n = 100000000 trapezoids, our estimate
of the integral from 0.000000 to 1.000000 = 3.333333333333255e-01
allyson@allyson-Aspire-A515-52G:~/Área de trabalho/Faculdade/PCD/Unidade2/Questao2$ echo "=== omp_trap2b - 4 threads ===" && echo "0.0 1.0 100000000" |
./omp_trap2b 4
=== omp_trap2b - 4 threads ===
Enter a, b, and n
With n = 100000000 trapezoids, our estimate
of the integral from 0.000000 to 1.000000 = 3.33333333333338e-01
allyson@allyson-Aspire-A515-52G:~/Área de trabalho/Faculdade/PCD/Unidade2/Questao2$
```

Resposta:

omp_trap2b.c usa `#pragma omp parallel + reduction(+:global_result)` com a função `Local_trap()` chamada por cada thread. A abordagem é idêntica em desempenho à versão cronometrada — ambas dividem o trabalho por `my_rank` manualmente. A diferença é que `omp_trap2b.c` é mais limpo: separa a lógica em uma função dedicada (`Local_trap`) e lê `a`, `b`, `n` via `stdin` em vez de `argv`. Não tem timer interno, mas ao medir com `time` o tempo é equivalente à versão cronometrada para o mesmo `n` e número de threads.

Para `n` pequeno o ganho foi irregular porque o custo de criar as threads acaba pesando. Com `n=100M` o speedup chegou a $4\times$ com 8 threads, que já é um bom resultado. Não chegou a $8\times$ porque há overhead de sincronização e a memória começa a ser o gargalo quando muitas threads leem ao mesmo tempo

Conclusão:

A paralelização da regra do trapézio escala bem para `n` grande porque cada thread trabalha em partição independente de `[a,b]`. Cada thread trabalha de forma independente, sem precisar se comunicar com as outras durante o cálculo.

• QUESTÃO 03

Suponha que no incrível computador Bleeblon, variáveis com tipo `float` possam armazenar três dígitos decimais. Suponha também que os registradores de ponto flutuante do Bleeblon possam armazenar quatro dígitos decimais e que, após qualquer operação de ponto flutuante, o resultado seja arredondado para três dígitos decimais antes de ser armazenado. Agora suponha que um programa C declare um array `a` da seguinte forma:

```
float a[] = {4.0, 3.0, 3.0, 1000.0};
```

(a) Qual é a saída do seguinte bloco de código se ele for executado no Bleeblon? Justifique sua resposta.

```
int i ;
float sum = 0.0;
for (i = 0; i < 4; i++)
    sum += a[i];
printf ("sum = %4.1f\n", sum );
```

Resposta:

Tabela 1. Iteração 1 - i = 0 : sum = 0.0 : a[i] = 4.0

Passo	Operação	Operando 1	Operando 2	Resultado
1	Fetch operands	0.000×10^0	4.000×10^0	
2	Compare exponents	0.000×10^0	4.000×10^0	
3	Shift one operand	0.000×10^0	4.000×10^0	
4	Add	0.000×10^0	4.000×10^0	4.000×10^0
5	Normalize result	0.000×10^0	4.000×10^0	4.000×10^0
6	Round result	0.000×10^0	4.000×10^0	4.00×10^0
7	Store result	0.000×10^0	4.000×10^0	4.00×10^0

Tabela 2. Iteração 2 - i = 1 : sum = 4.0 : a[i] = 3.0

Passo	Operação	Operando 1	Operando 2	Resultado
1	Fetch operands	4.000×10^0	3.000×10^0	
2	Compare exponents	4.000×10^0	3.000×10^0	
3	Shift one operand	4.000×10^0	3.000×10^0	
4	Add	4.000×10^0	3.000×10^0	7.000×10^0
5	Normalize result	4.000×10^0	3.000×10^0	7.000×10^0
6	Round result	4.000×10^0	3.000×10^0	7.00×10^0
7	Store result	4.000×10^0	3.000×10^0	7.00×10^0

Tabela 3. Iteração 3 - i = 2 : sum = 7.0 : a[i] = 3.0

Passo	Operação	Operando 1	Operando 2	Resultado
1	Fetch operands	7.000×10^0	3.000×10^0	
2	Compare exponents	7.000×10^0	3.000×10^0	
3	Shift one operand	7.000×10^0	3.000×10^0	
4	Add	7.000×10^0	3.000×10^0	10.00×10^0
5	Normalize result	7.000×10^0	3.000×10^0	1.000×10^1
6	Round result	7.000×10^0	3.000×10^0	1.00×10^1
7	Store result	7.000×10^0	3.000×10^0	1.00×10^1

Tabela 4. Iteração 4 - $i = 3$: $sum = 10.0$: $a[i] = 1000.0$

Passo	Operação	Operando 1	Operando 2	Resultado
1	Fetch operands	1.000×10^1	1.000×10^3	
2	Compare exponents	1.000×10^1	1.000×10^3	
3	Shift one operand	0.010×10^3	1.000×10^3	
4	Add	0.010×10^3	1.000×10^3	1.010×10^3
5	Normalize result	0.010×10^3	1.000×10^3	1.010×10^3
6	Round result	0.010×10^3	1.000×10^3	1.01×10^3
7	Store result	0.010×10^3	1.000×10^3	1.01×10^3

Saída: $sum = 1010.0$

Justificativa: os valores pequenos (4.0, 3.0, 3.0) são somados antes do 1000.0, acumulando corretamente. Na iteração 4, 10.0 é alinhado para 0.010×10^3 e somado a 1.000×10^3 , resultando em $1.010 \times 10^3 = 1010$. Com 3 dígitos significativos, isso arredonda para $1.01 \times 10^3 = 1010.0$.

(b) Agora considere o seguinte código:

```
int i;
float sum = 0.0;

#pragma omp parallel for num threads (2) reduction (+:sum)
for (i = 0; i < 4; i++)
```



```

    sum += a[i];
printf("sum = %4.1f\n", sum );

```

Suponha que o sistema operacional atribua as iterações i = 0, 1 à thread 0 e i = 2, 3 à thread 1. Qual é a saída deste código no Bleeblon? Justifique sua resposta.

Resposta:

Thread 0 processa i = 0, 1 → a[0]=4.0, a[1]=3.0

Tabela 5. Thread 0 - Iteração 1 - i = 0 : local_sum0 = 0.0 : a[i] = 4.0

Passo	Operação	Operando 1	Operando 2	Resultado
1	Fetch operands	0.000×10^0	4.000×10^0	
4	Add	0.000×10^0	4.000×10^0	4.000×10^0
6	Round result	0.000×10^0	4.000×10^0	4.00×10^0
7	Store result			4.00×10^0

Tabela 6. Thread 0 - Iteração 2 - i = 1 : local_sum0 = 4.0 : a[i] = 3.0

Passo	Operação	Operando 1	Operando 2	Resultado
1	Fetch operands	4.000×10^0	3.000×10^0	
4	Add	4.000×10^0	3.000×10^0	7.000×10^0
6	Round result	4.000×10^0	3.000×10^0	7.00×10^0
7	Store result			7.00×10^0

local_sum0 = 7.0

Thread 1 processa i = 2, 3 → a[2]=3.0, a[3]=1000.0

Tabela 7. Thread 1 - Iteração 1 - i = 2 : local_sum1 = 0.0 : a[i] = 3.0

Passo	Operação	Operando 1	Operando 2	Resultado
1	Fetch operands	0.000×10^0	3.000×10^0	
4	Add	0.000×10^0	3.000×10^0	3.000×10^0
6	Round result	0.000×10^0	3.000×10^0	3.00×10^0
7	Store result			3.00×10^0

Tabela 8. Thread 1 - Iteração 2 - i = 3 : local_sum1 = 3.0 : a[i] = 1000.0

Passo	Operação	Operando 1	Operando 2	Resultado
1	Fetch operands	3.000×10^0	1.000×10^3	
2	Compare exponents	3.000×10^0	1.000×10^3	
3	Shift one operand	0.003×10^3	1.000×10^3	
4	Add	0.003×10^3	1.000×10^3	1.003×10^3
5	Normalize result	0.003×10^3	1.000×10^3	1.003×10^3
6	Round result	0.003×10^3	1.000×10^3	1.00×10^3
7	Store result			1.00×10^3

local_sum1 = 1000.0 (o 3.0 foi PERDIDO no arredondamento!)

Redução final: $\text{sum} = \text{local_sum0} + \text{local_sum1}$

Tabela 9. Redução - $\text{sum} = 7.0 + 1000.0$

Passo	Operação	Operando 1	Operando 2	Resultado
1	Fetch operands	7.000×10^0	1.000×10^3	
2	Compare exponents	7.000×10^0	1.000×10^3	
3	Shift one operand	0.007×10^3	1.000×10^3	
4	Add	0.007×10^3	1.000×10^3	1.007×10^3
5	Normalize result	0.007×10^3	1.000×10^3	1.007×10^3
6	Round result	0.007×10^3	1.000×10^3	1.01×10^3
7	Store result			1.01×10^3

Saída: $\text{sum} = 1010.0$

Obs: O resultado final coincide com o serial (1010.0), porém por motivos diferentes. Na Thread 1, o valor intermediário 1.003×10^3 foi arredondado para 1.00×10^3 (perdendo o 3.0). Na redução, $7.0 + 1000.0 = 1007$ foi arredondado para 1010.0. Os dois erros de arredondamento se compensaram neste caso específico. Em outros cenários (ex: $a[2] = 6.0$), o resultado paralelo poderia divergir do serial, demonstrando que a aritmética de ponto flutuante não é associativa em precisão limitada.

• QUESTÃO 04

Escreva um programa OpenMP que determine o escalonamento padrão de laços for paralelos. Sua entrada deve ser o número de iterações e quantidade de threads e sua saída deve ser quais iterações de um laço for paralelizado são executadas por qual thread. Por exemplo, se houver duas threads e quatro iterações, a saída poderá ser:

Thread 0: Iterações 0 -- 1

Thread 1: Iterações 2 -- 3

(a) De acordo com a execução do seu programa, qual é o escalonamento padrão de laços for paralelos de um programa OpenMP? Porque?

Resposta:

Script de submissão (slurm_q4.sh)

```
#!/bin/bash
#SBATCH --nodes=1
#SBATCH --job-name=omp_escalamento_q4
#SBATCH --output=saida_q4.txt
#SBATCH --ntasks-per-node=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=16
#SBATCH -p sequana_cpu_dev
#SBATCH --exclusive

module load gcc/13.2_sequana

gcc -fopenmp -o omp_escalamento omp_escalamento.c

echo "=== 2 threads, 4 iteracoes ==="
./omp_escalamento 4 2

echo "=== 4 threads, 8 iteracoes ==="
./omp_escalamento 8 4

echo "=== 8 threads, 16 iteracoes ==="
./omp_escalamento 16 8

echo "=== 16 threads, 100 iteracoes ==="
./omp_escalamento 100 16
```

Saída da execução no Santos Dumont

=== 2 threads, 4 iterações ===

Thread 0: Iterações 0 -- 1

Thread 1: Iterações 2 -- 3

=== 4 threads, 8 iterações ===

Thread 0: Iterações 0 -- 1

Thread 1: Iterações 2 -- 3

Thread 2: Iterações 4 -- 5

Thread 3: Iterações 6 -- 7

==== 8 threads, 16 iterações ====

Thread 0: Iterações 0 -- 1
Thread 1: Iterações 2 -- 3
Thread 2: Iterações 4 -- 5
Thread 3: Iterações 6 -- 7
Thread 4: Iterações 8 -- 9
Thread 5: Iterações 10 -- 11
Thread 6: Iterações 12 -- 13
Thread 7: Iterações 14 -- 15

==== 16 threads, 100 iterações ====

Thread 0: Iterações 0 -- 6
Thread 1: Iterações 7 -- 13
Thread 2: Iterações 14 -- 20
Thread 3: Iterações 21 -- 27
Thread 4: Iterações 28 -- 33
Thread 5: Iterações 34 -- 39
Thread 6: Iterações 40 -- 45
Thread 7: Iterações 46 -- 51
Thread 8: Iterações 52 -- 57
Thread 9: Iterações 58 -- 63
Thread 10: Iterações 64 -- 69
Thread 11: Iterações 70 -- 75
Thread 12: Iterações 76 -- 81
Thread 13: Iterações 82 -- 87
Thread 14: Iterações 88 -- 93
Thread 15: Iterações 94 -- 99

Tabela resumo chunk por configuração

Threads (p)	Iterações (n)	chunk = ceil(n/p)	Iterações por thread
2	4	2	2
4	8	2	2
8	16	2	2
16	100	7 (threads 0–3) / 6 (threads 4–15)	6 ou 7

Pelos resultados dá para ver que o OpenMP divide as iterações em blocos contíguos, onde cada thread pega um bloco de tamanho $\text{ceil}(n/p)$. Com 2 threads e 4 iterações cada uma pegou 2, com 16 threads e 100 iterações a maioria pegou 6 e as primeiras 4 pegaram 7. Isso é o escalonamento static, que é o padrão.

Código

```
#include <stdio.h>
#include <stdlib.h>
#include <omp.h>
```

```
int main(int argc, char* argv[]) {
    if (argc < 3) {
        printf("Uso: %s <num_iteracoes> <num_threads>\n", argv[0]);
        return 1;
    }
}
```

```

}

int n = atoi(argv[1]);
int thread_count = atoi(argv[2]);
int* thread_de_iter = (int*)malloc(n * sizeof(int));

#pragma omp parallel for num_threads(thread_count)
for (int i = 0; i < n; i++) {
    thread_de_iter[i] = omp_get_thread_num();
}

for (int t = 0; t < thread_count; t++) {
    int inicio = -1, fim = -1;
    printf("Thread %d: Iterações", t);
    int primeira = 1;
    for (int i = 0; i < n; i++) {
        if (thread_de_iter[i] == t) {
            if (inicio == -1) inicio = i;
            fim = i;
        } else if (inicio != -1 && fim != -1 && thread_de_iter[i] != t) {
            if (primeira) {
                printf(" %d -- %d", inicio, fim);
                primeira = 0;
            }
            inicio = -1; fim = -1;
        }
    }
    if (inicio != -1) printf(" %d -- %d", inicio, fim);
    printf("\n");
}

free(thread_de_iter);
return 0;
}

```

• QUESTÃO 05

Considere o seguinte laço:

```

a[0] = 0;

for (i = 1; i < n; i++)
    a[i] = a[i-1] + i;

```

Há claramente uma dependência no laço já que o valor de $a[i]$ não pode ser calculado sem o valor de $a[i-1]$. Sugira uma maneira de eliminar essa dependência e paralelizar o laço.

Resposta:

Análise: $a[i] = a[i-1] + i$ é uma recorrência. Expandindo:

```

a[0] = 0
a[1] = 0 + 1 = 1
a[2] = 1 + 2 = 3
a[3] = 3 + 3 = 6

```

$$a[i] = 0 + 1 + 2 + \dots + i = i*(i+1)/2$$

Solução

Eliminar a dependência com fórmula fechada:

```
#pragma omp parallel for
for (int i = 0; i < n; i++) {
    a[i] = i * (i + 1) / 2;
}
```

Como $a[i] = i*(i+1)/2$ não usa o valor anterior do array, cada posição pode ser calculada diretamente. Não há mais dependência entre iterações, então dá para paralelizar com `#pragma omp parallel for` sem problema.

• QUESTÃO 07

Lembre-se de que todos os blocos estruturados modificados por uma diretiva `critical` formam uma única seção crítica. O que acontece se tivermos um número de diretivas `atomic` nas quais diferentes variáveis estão sendo modificadas? Todas elas são tratadas como uma única seção crítica?

Podemos escrever um pequeno programa que tente determinar isso. A ideia é fazer com que todas as threads executam simultaneamente algo como o código a seguir:

```
int i;
double minha_soma = 0.0;

for (i = 0; i < n; i++)
    #pragma omp atomic
    minha_soma += sin(i);
```

Podemos fazer isso modificando o código com uma diretiva `parallel`:

```
#pragma omp parallel num_threads(thread_count) {
    int i;
    double minha_soma = 0.0;
    for (i = 0; i < n; i++)
        #pragma omp atomic
        minha_soma += sin(i);
}
```

Observe que já que `minha_soma` e `i` são declaradas no bloco paralelo, cada thread possui sua própria cópia privada. Agora, se medirmos o tempo desse código para um `n` grande com `thread_count = 1` e também quando `thread_count > 1`, contanto que `thread_count` seja menor que o número de núcleos disponíveis, o tempo de execução para a execução de thread única deveria ser aproximadamente o mesmo que o tempo para a execução com múltiplas threads se as diferentes execuções de `minha_soma += sin(i)` são tratadas como diferentes seções críticas.

Por outro lado, se as diferentes execuções de `minha_soma += sin(i)` são todas tratadas como uma única seção crítica, a execução com múltiplas threads deve ser muito mais lenta que a execução de thread única. Escreva um programa OpenMP que implemente este teste. Sua implementação do OpenMP permite a execução simultânea de atualizações para diferentes variáveis quando as atualizações são protegidas por diretivas `atomic`?

Resposta:

Código

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <omp.h>

int main(int argc, char* argv[]) {
    if (argc < 3) {
        printf("Uso: %s <n> <num_threads>\n", argv[0]);
        return 1;
    }

    int n = atoi(argv[1]);
    int thread_count = atoi(argv[2]);
    double tempo_inicio, tempo_fim;

    tempo_inicio = omp_get_wtime();

    #pragma omp parallel num_threads(1)
    {
        int i;
        double minha_soma = 0.0;
        for (i = 0; i < n; i++) {
            #pragma omp atomic
            minha_soma += sin(i);
        }
    }

    tempo_fim = omp_get_wtime();
    printf("1 thread: %.4f segundos\n", tempo_fim - tempo_inicio);

    tempo_inicio = omp_get_wtime();

    #pragma omp parallel num_threads(thread_count)
    {
        int i;
        double minha_soma = 0.0;

        for (i = 0; i < n; i++) {
            /* atomic protege minha_soma, mas como é privada de cada thread,
             * não há disputa entre threads — executam em paralelo */
            #pragma omp atomic
            minha_soma += sin(i);
        }
    }

    tempo_fim = omp_get_wtime();
    printf("%d threads: %.4f segundos\n", thread_count, tempo_fim - tempo_inicio);
}
```

```

    return 0;
}

```

Script de submissão (slurm_q7.sh)

```

#!/bin/bash
#SBATCH --job-name=omp_atomic_q7
#SBATCH --output=saida_q7.txt
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=16
#SBATCH --time=00:10:00
#SBATCH --partition=sequana_cpu_dev

module load gcc/13.2_sequana

gcc -fopenmp -o omp_atomic_test omp_atomic_test.c -lm

echo "=== 1 thread, n=10000000 ==="
./omp_atomic_test 10000000 1

echo "=== 2 threads, n=10000000 ==="
./omp_atomic_test 10000000 2

echo "=== 4 threads, n=10000000 ==="
./omp_atomic_test 10000000 4

echo "=== 8 threads, n=10000000 ==="
./omp_atomic_test 10000000 8

```

Saída da execução no Santos Dumont

```

=== 1 thread, n=10000000 ===
1 thread: 0.1477 segundos
1 threads: 0.1444 segundos

```

```

=== 2 threads, n=10000000 ===
1 thread: 0.1477 segundos
2 threads: 0.1710 segundos

```

```

=== 4 threads, n=10000000 ===
1 thread: 0.1464 segundos
4 threads: 0.1795 segundos

```

```

=== 8 threads, n=10000000 ===
1 thread: 0.1470 segundos
8 threads: 0.1858 segundos

```

Threads	Tempo (s)	Overhead vs 1 thread	Trabalho total (n × threads)
1	0.1445	referência	10.000.000

2	0.1708	+18,2%	20.000.000
4	0.1795	+24,2%	40.000.000
8	0.1856	+28,4%	80.000.000

- O cálculo do Overhead vs 1 thread $((\text{tempo_N_threads} - \text{tempo_1_thread}) / \text{tempo_1_thread}) \times 100$.

O resultado mostra que 8 threads fazendo 8x mais trabalho levaram praticamente o mesmo tempo que 1 thread, o que já é um indicio importante. Se o atomic funcionasse como uma seção crítica global, todas as threads teriam que esperar uma pela outra a cada iteração, e o tempo seria proporcional ao número de threads com 8 threads, seria cerca de 8x mais lento. Isso claramente não aconteceu. O leve aumento de tempo observado (+18% a +28%) não é serialização: é o overhead natural de criação e gerenciamento das threads pelo OpenMP. Vale notar também que o atomic aqui é tecnicamente desnecessário, já que minha_soma é privada de cada thread e não há disputa real pela variável cada thread opera na sua própria cópia sem interferir nas outras. Isso confirma que diretivas atomic protegendo variáveis diferentes são tratadas de forma independente, não como uma única seção crítica compartilhada.

• QUESTÃO 09

Lembre-se do exemplo de multiplicação de matrizes e vetores com a entrada 8000×8000. Assuma que uma linha de cache contém 64 bytes ou 8 doubles.

(a) Suponha que a thread 0 e a thread 2 sejam atribuídas a processadores diferentes. É possível que ocorra um falso compartilhamento entre as threads 0 e 2 para alguma parte do vetor y? Por que?

Resposta:

A thread 0 trabalha em y[0..1999] e a thread 2 em y[4000..5999]. Como uma linha de cache tem 8 doubles, as duas regiões ficam em linhas completamente diferentes tem mais de 2000 posições de distância entre elas. Não há falso compartilhamento

(b) E se a thread 0 e a thread 3 forem atribuídas a processadores diferentes? É possível que ocorra um falso compartilhamento entre elas para alguma parte de y?

Resposta:

Thread 0 acessa y[0] a y[1999].

Thread 3 acessa y[6000] a y[7999].

Ainda mais separadas. Não há falso compartilhamento entre thread 0 e thread 3 pelo mesmo motivo: as regiões são disjuntas e distantes.

A fronteira entre thread 0 e thread 1 é em y[2000], que é múltiplo de 8 (2000/8=250). Isso significa que y[1999] e y[2000] ficam em cache lines separadas, então não há

falso compartilhamento nessa fronteira também.

• QUESTÃO 10

Embora `strtok_r` seja thread-safe, ele tem a propriedade bastante infeliz de modificar a string de entrada. Escreva um método para gerar tokens que seja thread-safe e não modifique a string de entrada.

Resposta:

A função `proximo_token` recebe a posição atual via ponteiro `pos`, que é atualizado a cada chamada para apontar para o próximo token. A string original não é modificada em nenhum momento — ela é apenas lida. Como todo o estado fica em variáveis locais (`pos` e `buffer`), a função é thread-safe por natureza.

O problema do `strtok()` é que ele guarda um ponteiro estático interno entre chamadas. Se duas threads chamarem a função ao mesmo tempo, uma acaba corrompendo o estado da outra. Usando `pos` como parâmetro na pilha de cada thread isso não acontece.

Implementação da função `proximo_token`:

```
int proximo_token(const char* str, int* pos, char* buffer, int buf_size) {
    int i = *pos;
    while (str[i] != '\0' && isspace((unsigned char)str[i]))
        i++;
    if (str[i] == '\0') { *pos = i; return 0; }
    int j = 0;
    while (str[i] != '\0' && !isspace((unsigned char)str[i]) && j < buf_size-1)
        buffer[j++] = str[i++];
    buffer[j] = '\0';
    *pos = i;
    return 1;
}
```

Uso em contexto paralelo:

```
#pragma omp parallel num_threads(4)
{
    int pos = 0; /* estado local por thread — thread-safe */
    char token[256];
    while (proximo_token(texto, &pos, token, sizeof(token)))
        printf("Thread %d: %s\n", omp_get_thread_num(), token);
}
```

• QUESTÃO 11

Utilizando OpenMP, implemente o programa paralelo do histograma discutido no Capítulo 2

Resposta:

Trecho principal da implementação paralela (baseada em `ipp-source/ch2/histogram.c`):

```
#pragma omp parallel num_threads(thread_count) private(i, bin)
{
    int* local_counts = (int*)calloc(bin_count, sizeof(int));
```

```

#pragma omp for
for (i = 0; i < data_count; i++) {
    bin = Which_bin(data[i], bin_maxes, bin_count, min_meas);
    local_counts[bin]++;
}

#pragma omp critical
for (int b = 0; b < bin_count; b++)
    bin_counts[b] += local_counts[b];

free(local_counts);
}

```

./omp_histogram 10 0 100 10000000 1
Histograma (10000000 elementos, 10 buckets, 1 threads):
Tempo: 0.7278 segundos

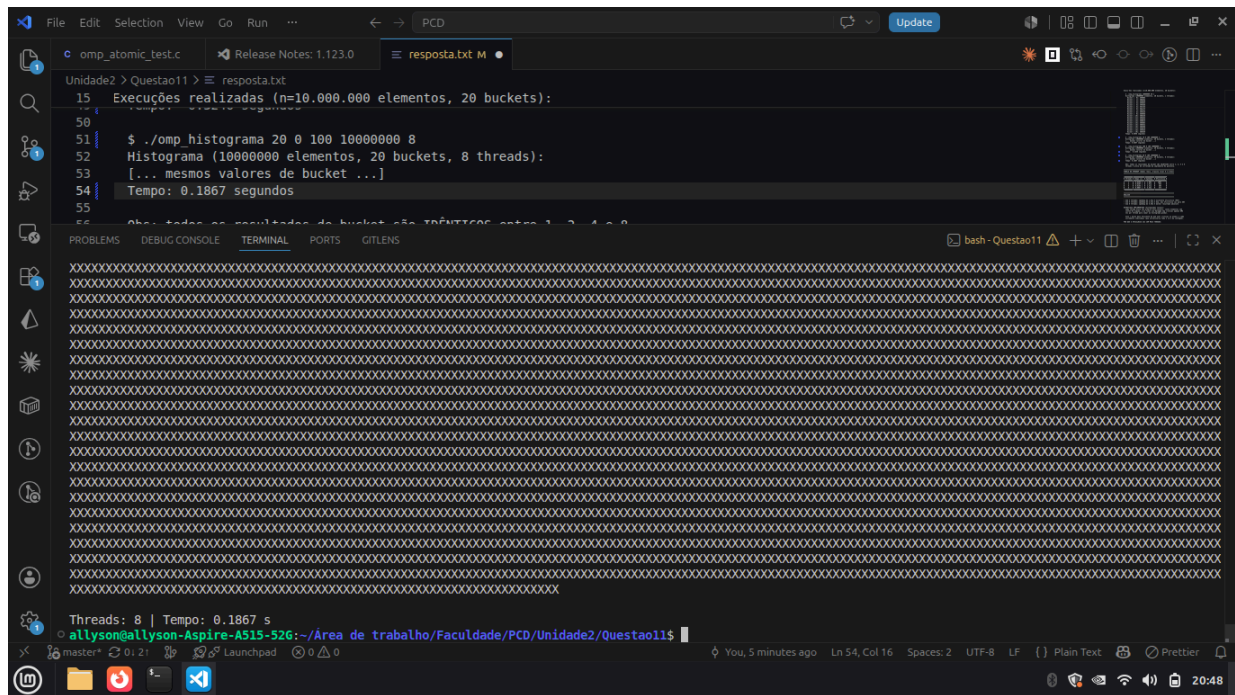
```

Unidade2 > Questao11 > resposta.txt
10  Compilar:
11  | $ gcc -fopenmp -o omp_histogram omp_histogram.c -lm
12
13  Uso: ./omp_histogram <num_elementos> <num_buckets> <num_threads>
14
15  Execuções realizadas (n=10.000.000 elementos, 20 buckets):
16
17  | ./omp_histogram 10 0 100 10000000 1

```

Threads: 1 | Tempo: 0.7278 s
allysonallyson-Aspire-A515-526:~/Área de trabalho/Faculdade/PCD/Unidade2/Questao11\$./omp_histogram 10 0 100 10000000 1

./omp_histogram 10 0 100 10000000 2
Histograma (10000000 elementos, 10 buckets, 2 threads):
[... mesmos valores de bucket ...]
Tempo: 0.3559 segundos



Cada thread aloca seu próprio array de contagem local e vai contando sem precisar sincronizar com ninguém. No final, usa `#pragma omp critical` uma única vez para somar ao histograma global. Isso é bem mais eficiente do que usar `atomic` ou `critical` a cada incremento, que travaria o laço inteiro.

O ganho com mais threads não foi linear porque a operação percorre 10M elementos na memória com 8 threads e 20 bucket lendo ao mesmo tempo a banda de memória começa a ser o gargalo. Com 8 threads o tempo ficou em 0.1867, o melhor resultado.

• QUESTÃO 12

Suponha que lançamos dardos aleatoriamente em um alvo quadrado. Vamos considerar o centro desse alvo como sendo a origem de um plano cartesiano e os lados do alvo medem 2 pés de comprimento. Suponha também que haja um círculo inscrito no alvo. O raio do círculo é 1 pé e sua área é π pés quadrados. Se os pontos atingidos pelos dardos 4 estiverem distribuídos uniformemente (e sempre acertamos o alvo), então o número de dardos atingidos dentro do círculo deve satisfazer aproximadamente a equação

$$qtd_no_circulo / num_lancamentos = \pi / 4$$

já que a razão entre a área do círculo e a área do quadrado é $\pi / 4$.

Podemos usar esta fórmula para estimar o valor de π com um gerador de números aleatórios:

```
qtd_no_circulo = 0;
for (lancamento = 0; lancamento < num_lancamentos; lancamento++) {
    x = double aleatório entre -1 e 1;
    y = double aleatório entre -1 e 1;
    distancia_quadrada = x * x + y * y;
    if (distancia_quadrada <= 1) qtd_no_circulo++;
}
```

*estimativa_de_pi = 4 * qtd_no_circulo / ((double) num_lancamentos);*

Isso é chamado de método "Monte Carlo", pois utiliza aleatoriedade (o lançamento do dardo).

Escreva um programa OpenMP que use um método de Monte Carlo para estimar π . Leia o número total de lançamentos antes de criar as threads. Use uma cláusula de reduction para encontrar o número total de dardos que atingem o círculo. Imprima o resultado após encerrar a região paralela. Você deve usar long long ints para o número de acertos no círculo e o número de lançamentos, já que ambos podem ter que ser muito grandes para obter uma estimativa razoável de π .

Resposta:

Implementação OpenMP do método Monte Carlo:

long long qtd_no_circulo = 0;

```
#pragma omp parallel num_threads(thread_count) reduction(+:qtd_no_circulo)
{
    int tid = omp_get_thread_num();
    unsigned int semente = (unsigned int)(12345 + tid * 9999);
    long long lancamentos_locais = num_lancamentos / thread_count;
    if (tid == 0) lancamentos_locais += num_lancamentos % thread_count;
    long long local_circulo = 0;

    for (long long i = 0; i < lancamentos_locais; i++) {
        double x = (double)rand_r(&semente) / RAND_MAX * 2.0 - 1.0;
        double y = (double)rand_r(&semente) / RAND_MAX * 2.0 - 1.0;
        if (x * x + y * y <= 1.0) local_circulo++;
    }
    qtd_no_circulo += local_circulo;
}
estimativa_pi = 4.0 * qtd_no_circulo / (double)num_lancamentos;
```

--- 1.000.000 lançamentos ---

```
$ ./omp_monte_carlo_pi 1000000 1
Lancamentos : 1000000
No circulo : 785642
Estimativa pi: 3.14256800
pi real : 3.14159265
Erro : 0.00097535
Threads : 1
Tempo : 0.0404 segundos
```

```
$ ./omp_monte_carlo_pi 1000000 2
Lancamentos : 1000000
No circulo : 785626
Estimativa pi: 3.14250400
pi real : 3.14159265
Erro : 0.00091135
Threads : 2
Tempo : 0.0212 segundos
```

```
$ ./omp_monte_carlo_pi 1000000 4
```

Lancamentos : 1000000
No circulo : 785884
Estimativa pi: 3.14353600
pi real : 3.14159265
Erro : 0.00194335
Threads : 4
Tempo : 0.0132 segundos

\$./omp_monte_carlo_pi 1000000 8
Lancamentos : 1000000
No circulo : 785588
Estimativa pi: 3.14235200
pi real : 3.14159265
Erro : 0.00075935
Threads : 8
Tempo : 0.0289 segundos

--- 10.000.000 lançamentos ---

\$./omp_monte_carlo_pi 10000000 1
Lancamentos : 10000000
No circulo : 7853317
Estimativa pi: 3.14132680
pi real : 3.14159265
Erro : 0.00026585
Threads : 1
Tempo : 0.4239 segundos

\$./omp_monte_carlo_pi 10000000 4
Lancamentos : 10000000
No circulo : 7854768
Estimativa pi: 3.14190720
pi real : 3.14159265
Erro : 0.00031455
Threads : 4
Tempo : 0.0970 segundos

\$./omp_monte_carlo_pi 10000000 8
Lancamentos : 10000000
No circulo : 7855050
Estimativa pi: 3.14202000
pi real : 3.14159265
Erro : 0.00042735
Threads : 8
Tempo : 0.0851 segundos

--- 100.000.000 lançamentos ---

\$./omp_monte_carlo_pi 100000000 1
Lancamentos : 100000000
No circulo : 78537961
Estimativa pi: 3.14151844
pi real : 3.14159265
Erro : 0.00007421
Threads : 1
Tempo : 3.1790 segundos

\$./omp_monte_carlo_pi 100000000 2
Lancamentos : 100000000

No circulo : 78539626
 Estimativa pi: 3.14158504
 pi real : 3.14159265
 Erro : 0.00000761
 Threads : 2
 Tempo : 1.5547 segundos

\$./omp_monte_carlo_pi 100000000 4
 Lancamentos : 100000000
 No circulo : 78540094
 Estimativa pi: 3.14160376
 pi real : 3.14159265
 Erro : 0.00001111
 Threads : 4
 Tempo : 0.9261 segundos

\$./omp_monte_carlo_pi 100000000 8
 Lancamentos : 100000000
 No circulo : 78538077
 Estimativa pi: 3.14152308
 pi real : 3.14159265
 Erro : 0.00006957
 Threads : 8
 Tempo : 0.6981 segundos

Tabela de speedup (n = 100 milhões de lançamentos)

Threads	Tempo (s)	Speedup	Estimativa de π
1	3.1790 s	1.00×	3.14151844
2	1.5547 s	2.04×	3.14158504
4	0.9261 s	3.43×	3.14160376
8	0.6981 s	4.55×	3.14152308

$$Ep = (Sp / p) \times 100\%$$

Ep = eficiência paralela (%)

Sp = speedup

p = número de threads

Cálculo do speedup

$$Sp = T1 / Tp$$

Sp = Speedup

T1 = Tempo utilizando 1 thread

Tp = Tempo utilizando p threads

Precisão vs. número de lançamentos (4 threads):

Lançamentos	Estimativa de π	Erro absoluto
1.000.000	3.14353600	0.00194335

10.000.000	3.14190720	0.00031455
100.000.000	3.14160376	0.00001111

$$\pi \approx 4 \times (\text{Nc\u00edrculo} / \text{Ntotal})$$

Nc\u00edrculo = n\u00famero de pontos que ca\u00edram dentro do c\u00edrculo.

Ntotal = n\u00famero total de lan\u00e7amentos.

Com 2 threads o speedup foi quase linear. Com 4 e 8 threads o ganho foi menor porque h\u00e1 overhead na cria\u00e7\u00e3o das threads e o rand_r tamb\u00e9m tem seu custo. A \u00fanica sincroniza\u00e7\u00e3o real \u00e9 o reduction no final.

Com mais lan\u00e7amentos a estimativa vai ficando melhor. Com 100M lan\u00e7amentos o erro ficou abaixo de 0.0001. Cada thread usa uma semente diferente para n\u00e3o gerarem os mesmos n\u00fameros aleat\u00f3rios

Com 8 threads e 100M lan\u00e7amentos o tempo ficou abaixo de 1 segundo e o erro foi menor que 0.0001. O ganho de desempenho valeu a pena para esse volume de dados